

EXPERIMENT 8

Student Name: Husanpreet Kaur

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70138

Section/Group: 24AIT_KRG-G2

Date: 24-08-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To study and implement PostgreSQL Views and Materialized Views for simplifying data access and analysis by combining data from multiple tables, performing SQL operations such as joins and aggregations, controlling access to data, and analyzing query execution performance.

Question 1: A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount.

For every order, first calculate the Gross Amount using:

Gross Amount = Unit Price × Quantity

Then calculate the Discount Amount using:

Discount Amount = Gross Amount × Discount Percentage / 100

Finally, calculate the Final Amount using:

Final Amount = Gross Amount – Discount Amount

Create a complex view using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user only SELECT permission on the view. The client must

EXPERIMENT 8

not have direct access to the three original tables and must not be allowed to modify the view data.

Solution: CREATE TABLE customer_master

```
(  
    customer_id VARCHAR(5) PRIMARY KEY,  
    full_name VARCHAR(50) NOT NULL,  
    city VARCHAR(30)  
);
```

CREATE TABLE product_catalog

```
(  
    product_id VARCHAR(5) PRIMARY KEY,  
    product_name VARCHAR(50) NOT NULL,  
    unit_price NUMERIC(10,2) NOT NULL  
);
```

CREATE TABLE sales_orders

```
(  
    order_id SERIAL PRIMARY KEY,  
    product_id VARCHAR(5)  
        REFERENCES product_catalog(product_id),  
    quantity INT NOT NULL,  
    customer_id VARCHAR(5)  
        REFERENCES customer_master(customer_id),
```

EXPERIMENT 8

```
discount_percent NUMERIC(5,2),  
order_date DATE NOT NULL  
);
```

```
INSERT INTO customer_master  
(customer_id, full_name, city)  
VALUES  
( 'C1', 'Amit Sharma', 'Delhi'),  
( 'C2', 'Priya Verma', 'Mumbai'),  
( 'C3', 'Ravi Kumar', 'Bangalore'),  
( 'C4', 'Neha Singh', 'Kolkata'),  
( 'C5', 'Arjun Mehta', 'Hyderabad');
```

```
INSERT INTO product_catalog  
(product_id, product_name, unit_price)  
VALUES  
( 'P1', 'Smartphone X100', 25000),  
( 'P2', 'Laptop Pro 15', 65000),  
( 'P3', 'Wireless Earbuds', 5000),  
( 'P4', 'Smartwatch Fit', 30000),  
( 'P5', 'Gaming Console', 45000);
```

```
INSERT INTO sales_orders  
(product_id, quantity, customer_id, discount_percent, order_date)
```

EXPERIMENT 8

VALUES

```
('P1', 2, 'C1', 5, '2025-09-01'),  
( 'P2', 1, 'C2', 10, '2025-09-02'),  
( 'P3', 3, 'C3', 0, '2025-09-03'),  
( 'P4', 1, 'C4', 8, '2025-09-04'),  
( 'P5', 1, 'C5', 12, '2025-09-05'),  
( 'P1', 1, 'C2', 5, '2025-09-06'),  
( 'P3', 2, 'C1', 0, '2025-09-07');
```

---EXPLAIN ANALYZE---

CREATE VIEW vw_sales_order_details AS

SELECT

```
so.order_id,  
so.order_date,  
cm.full_name,  
pc.product_name,
```

-- Gross Amount

```
pc.unit_price * so.quantity AS gross_amount,
```

-- Discount Amount

```
(pc.unit_price * so.quantity)  
* so.discount_percent / 100 AS discount_amount,
```

EXPERIMENT 8

-- Final Amount

(pc.unit_price * so.quantity)

- ((pc.unit_price * so.quantity)

* so.discount_percent / 100) AS final_amount

FROM sales_orders so

JOIN customer_master cm

ON so.customer_id = cm.customer_id

JOIN product_catalog pc

ON so.product_id = pc.product_id;

SELECT *

FROM vw_sales_order_details;

CREATE USER client_user

PASSWORD '12345';

REVOKE ALL ON customer_master FROM client_user;

REVOKE ALL ON product_catalog FROM client_user;

REVOKE ALL ON sales_orders FROM client_user;

GRANT SELECT

EXPERIMENT 8

ON vw_sales_order_details

TO client_user;

SELECT *

FROM vw_sales_order_details;

OUTPUT:

Data Output Messages Notifications							
Showing rows: 1 to 7 Page No: 1 of 1							
	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	gross_amount numeric	discount_amount numeric	final_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	50000.00	2500.0000000000000000	47500.0000000000000000
2	2	2025-09-02	Priya Verma	Laptop Pro 15	65000.00	6500.0000000000000000	58500.0000000000000000
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00	0.00000000000000000000	15000.0000000000000000
4	4	2025-09-04	Neha Singh	Smartwatch Fit	30000.00	2400.0000000000000000	27600.0000000000000000
5	5	2025-09-05	Arjun Mehta	Gaming Console	45000.00	5400.0000000000000000	39600.0000000000000000
6	6	2025-09-06	Priya Verma	Smartphone X100	25000.00	1250.0000000000000000	23750.0000000000000000
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00	0.00000000000000000000	10000.0000000000000000

Question 2: An e-commerce company wants to regularly analyze the performance of its product categories. The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL Materialized View named mv_category_sales to store the result of this analytical query.

The report must contain the following attributes:

- Category ID
- Category Name

EXPERIMENT 8

- Total Products Sold
- Total Orders
- Unique Customers
- Total Revenue
- Average Revenue per Order
- Category Performance

Solution: CREATE TABLE category_master (
 category_id VARCHAR(5) PRIMARY KEY,
 category_name VARCHAR(50) NOT NULL
);

CREATE TABLE product_master (
 product_id VARCHAR(5) PRIMARY KEY,
 product_name VARCHAR(50) NOT NULL,
 category_id VARCHAR(5)
 REFERENCES category_master(category_id)
);

CREATE TABLE customer_master (
 customer_id VARCHAR(5) PRIMARY KEY,
 customer_name VARCHAR(50) NOT NULL
);

CREATE TABLE order_details (
 order_id VARCHAR(5) PRIMARY KEY,
 customer_id VARCHAR(5)

EXPERIMENT 8

```
REFERENCES customer_master(customer_id),  
product_id VARCHAR(5)  
REFERENCES product_master(product_id),  
quantity INT NOT NULL,  
unit_price NUMERIC(10,2) NOT NULL  
);
```

```
INSERT INTO category_master  
(category_id, category_name)  
VALUES  
( 'C1', 'Electronics'),  
( 'C2', 'Clothing'),  
( 'C3', 'Books');
```

```
INSERT INTO product_master  
(product_id, product_name, category_id)  
VALUES  
( 'P101', 'Laptop', 'C1'),  
( 'P102', 'Headphones', 'C1'),  
( 'P201', 'Jacket', 'C2'),  
( 'P202', 'Shoes', 'C2'),  
( 'P301', 'SQL Book', 'C3');
```

```
INSERT INTO customer_master
```

EXPERIMENT 8

(customer_id, customer_name)

VALUES

('U1', 'Aman'),

('U2', 'Priya'),

('U3', 'Rahul'),

('U4', 'Simran');

INSERT INTO order_details

(order_id, customer_id, product_id, quantity, unit_price)

VALUES

('O101', 'U1', 'P101', 1, 5000),

('O102', 'U2', 'P102', 2, 1000),

('O103', 'U1', 'P201', 1, 3000),

('O104', 'U3', 'P202', 2, 2000),

('O105', 'U4', 'P301', 3, 500),

('O106', 'U2', 'P101', 1, 5000),

('O107', 'U3', 'P201', 1, 3000),

('O108', 'U4', 'P102', 1, 1000);

---ORIGINAL ANALYTICAL QUERY---

SELECT

c.category_id,

c.category_name,

SUM(od.quantity) AS total_products_sold,

EXPERIMENT 8

```
COUNT(DISTINCT od.order_id) AS total_orders,
COUNT(DISTINCT od.customer_id) AS unique_customers,
SUM(od.quantity * od.unit_price) AS total_revenue,
ROUND(
    SUM(od.quantity * od.unit_price)::NUMERIC
    / COUNT(DISTINCT od.order_id),
    2
) AS avg_revenue_per_order,
CASE
    WHEN SUM(od.quantity * od.unit_price) >= 10000
        THEN 'Excellent'
    WHEN SUM(od.quantity * od.unit_price) >= 6000
        THEN 'Good'
    WHEN SUM(od.quantity * od.unit_price) >= 3000
        THEN 'Average'
    ELSE 'Low'
END AS category_performance
FROM category_master c
JOIN product_master p
    ON c.category_id = p.category_id
JOIN order_details od
    ON p.product_id = od.product_id
GROUP BY
    c.category_id,
```

EXPERIMENT 8

c.category_name

ORDER BY

c.category_id;

---EXPLAIN ANALYZE ORIGINAL QUERY---

EXPLAIN ANALYZE

SELECT

c.category_id,

c.category_name,

SUM(od.quantity) AS total_products_sold,

COUNT(DISTINCT od.order_id) AS total_orders,

COUNT(DISTINCT od.customer_id) AS unique_customers,

SUM(od.quantity * od.unit_price) AS total_revenue,

ROUND(

SUM(od.quantity * od.unit_price)::NUMERIC

/ COUNT(DISTINCT od.order_id),

2

) AS avg_revenue_per_order,

CASE

WHEN SUM(od.quantity * od.unit_price) >= 10000

THEN 'Excellent'

WHEN SUM(od.quantity * od.unit_price) >= 6000

THEN 'Good'

WHEN SUM(od.quantity * od.unit_price) >= 3000

EXPERIMENT 8

```
        THEN 'Average'
    ELSE 'Low'
END AS category_performance
FROM category_master c
JOIN product_master p
    ON c.category_id = p.category_id
JOIN order_details od
    ON p.product_id = od.product_id
GROUP BY
    c.category_id,
    c.category_name
ORDER BY
    c.category_id;

CREATE MATERIALIZED VIEW mv_category_sales_01 AS
SELECT
    c.category_id,
    c.category_name,
    SUM(od.quantity) AS total_products_sold,
    COUNT(DISTINCT od.order_id) AS total_orders,
    COUNT(DISTINCT od.customer_id) AS unique_customers,
    SUM(od.quantity * od.unit_price) AS total_revenue,
    ROUND(
        SUM(od.quantity * od.unit_price)::NUMERIC
```

EXPERIMENT 8

```
/ COUNT(DISTINCT od.order_id),  
2  
) AS avg_revenue_per_order,  
CASE  
    WHEN SUM(od.quantity * od.unit_price) >= 10000  
        THEN 'Excellent'  
    WHEN SUM(od.quantity * od.unit_price) >= 6000  
        THEN 'Good'  
    WHEN SUM(od.quantity * od.unit_price) >= 3000  
        THEN 'Average'  
    ELSE 'Low'  
END AS category_performance  
FROM category_master c  
JOIN product_master p  
    ON c.category_id = p.category_id  
JOIN order_details od  
    ON p.product_id = od.product_id  
GROUP BY  
    c.category_id,  
    c.category_name;  
  
---DISPLAY MATERIALIZED VIEW---  
SELECT *  
FROM mv_category_sales_01
```

EXPERIMENT 8

ORDER BY category_id;

---EXPLAIN ANALYZE MATERIALIZED VIEW---

EXPLAIN ANALYZE

SELECT *

FROM mv_category_sales_01

ORDER BY category_id;

INSERT INTO order_details

(order_id, customer_id, product_id, quantity, unit_price)

VALUES

('O109', 'U1', 'P101', 2, 5000);

---DISPLAY BEFORE REFRESH---

SELECT *

FROM mv_category_sales_01

ORDER BY category_id;

---REFRESH MATERIALIZED VIEW---

REFRESH MATERIALIZED VIEW mv_category_sales_01;

---DISPLAY AFTER REFRESH---

SELECT *

FROM mv_category_sales_01

ORDER BY category_id;

EXPLAIN ANALYZE

SELECT *

FROM mv_category_sales_01

ORDER BY category_id;

OUTPUT:

Data Output

Messages

Notifications

☰

📄

▼

📋

▼

🗑️

🔍

⬇️

📈

SQL

Showing rows: 1 to 3

✎

Page No: 1 of 1

	category_id [PK] character varying (5) ✎	category_name character varying (50) ✎	total_products_sold bigint 🔒	total_orders bigint 🔒	unique_customers bigint 🔒	total_revenue numeric 🔒	avg_revenue_per_order numeric 🔒	category_performance text 🔒
1	C1	Electronics	5	4	3	13000.00	3250.00	Excellent
2	C2	Clothing	4	3	2	10000.00	3333.33	Excellent
3	C3	Books	3	1	1	1500.00	1500.00	Low

Data Output	Messages	Notifications
+ 📄 ▼ 📁 ▼ 🗑️ 💾 ⬇️ 📈 SQL S		
	QUERY PLAN text 🔒	
1	GroupAggregate (cost=92.65..119.45 rows=480 width=262) (actual time=0.231..0.249 rows=3 loops=1)	
2	Group Key: c.category_id	
3	-> Sort (cost=92.65..94.35 rows=680 width=210) (actual time=0.192..0.195 rows=8 loops=1)	
4	Sort Key: c.category_id, od.order_id	
5	Sort Method: quicksort Memory: 25kB	
6	-> Hash Join (cost=40.25..60.65 rows=680 width=210) (actual time=0.086..0.096 rows=8 loops=1)	
7	Hash Cond: ((p.category_id)::text = (c.category_id)::text)	
8	-> Hash Join (cost=19.45..38.06 rows=680 width=92) (actual time=0.047..0.054 rows=8 loops=1)	
9	Hash Cond: ((od.product_id)::text = (p.product_id)::text)	
10	-> Seq Scan on order_details od (cost=0.00..16.80 rows=680 width=92) (actual time=0.008..0.009 rows=8 loops=1)	
11	-> Hash (cost=14.20..14.20 rows=420 width=48) (actual time=0.022..0.023 rows=5 loops=1)	
12	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
13	-> Seq Scan on product_master p (cost=0.00..14.20 rows=420 width=48) (actual time=0.009..0.010 rows=5 loops=1)	
14	-> Hash (cost=14.80..14.80 rows=480 width=142) (actual time=0.029..0.030 rows=3 loops=1)	
15	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
16	-> Seq Scan on category_master c (cost=0.00..14.80 rows=480 width=142) (actual time=0.019..0.020 rows=3 loops=1)	
17	Planning Time: 0.489 ms	
18	Execution Time: 0.344 ms	

EXPERIMENT 8

Data Output	Messages	Notifications
≡ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	QUERY PLAN text 🔒	
1	Sort (cost=24.18..24.88 rows=280 width=262) (actual time=0.057..0.059 rows=3 loops=1)	
2	Sort Key: category_id	
3	Sort Method: quicksort Memory: 25kB	
4	-> Seq Scan on mv_category_sales_01 (cost=0.00..12.80 rows=280 width=262) (actual time=0.025..0.027 rows=3 loo...	
5	Planning Time: 0.179 ms	
6	Execution Time: 0.088 ms	

LEARNING OUTCOMES:

1. Understood PostgreSQL Views and Materialized Views and explain their purpose and differences.
2. Created and managed Views using SQL queries involving multiple tables, joins, calculations, and filtering.
3. Created Materialized Views to store precomputed results of complex queries involving aggregation and analysis.
4. Performed data analysis using SQL functions such as SUM(), COUNT(), ROUND(), GROUP BY, and conditional CASE statements.
5. Analyzed query performance using EXPLAIN ANALYZE and understand how Materialized Views can improve data retrieval for analytical queries.